

YEREVAN STATE UNIVERSITY
FACULTY OF MATHEMATICS AND MECHANICS
DEPARTMENT OF PROBABILITY THEORY AND
MATHEMATICAL STATISTICS

APPLIED STATISTICS AND DATA SCIENCE
MASTER'S PROGRAM

HAYK MINASYAN

MASTER'S THESIS

Advancing 3D Reconstruction Using 3D Foundation Models

*For the purpose of applying for the Master's degree qualification in the field of
"Statistics"*

YEREVAN 2026

CONTENT

Abstract	3
1 Introduction	4
2 Related Work	5
2.1 DUS3R as a Foundation for 3D Reconstruction	5
3 Methodology	8
3.1 Gaussian Blur Degradation	8
3.1.1 Training objectives	9
3.1.2 Data Description	10
3.1.3 Results	11
3.2 Motion and Defocus Blur Degradation	12
3.2.1 Training objectives	13
3.2.2 Data description	15
3.2.3 Results	15
Conclusions and Recommendations	18
References	20

ABSTRACT

This Master’s thesis addresses 3D reconstruction from multi-view images, building on strong geometric foundation models of the DUS_t3R type. The work focuses on improving the stability of reconstruction when input images are degraded by Gaussian, motion or defocus blur. It proposes lightweight modules that preprocess RGB inputs for geometric estimation, trained by knowledge distillation from strong teacher model, as well as the joint fine-tuning of the integrated module and DUS_t3R architecture. Experiments are conducted on subsets of the CO3D dataset, using sequence-level splits and synthetic degradations.

1. INTRODUCTION

Three-dimensional (3D) reconstruction from images is an important topic in computer vision because it is used in robotics, autonomous systems, AR/VR, etc. In recent years, 3D foundation models such as DUS3R have shown strong results on clean images. However, in real conditions, images are often degraded by Gaussian blur, motion blur, defocus blur. These degradations reduce geometric consistency and lower reconstruction quality. For this reason, improving robustness to degraded inputs is both scientifically relevant and practically useful.

The purpose of this thesis is to improve the robustness of the DUS3R-based 3D reconstruction pipeline under degraded image conditions. To achieve this purpose, the study has several objectives: first, to analyze how baseline DUS3R behaves under blur, motion blur, defocus images; second, to integrate restoration front-end models with DUS3R and improve the combined architecture through joint fine-tuning or knowledge distillation from stronger teacher models; and fourth, to compare baseline, pretrained-front-end, and finetuned-front-end settings using a consistent evaluation protocol based on Chamfer Distance.

The theoretical foundation of the thesis includes deep learning for geometric vision, multi-view 3D reconstruction, and image restoration. The methodological foundation includes controlled degradation generation, supervised fine-tuning, knowledge distillation comparative experiments across model variants, and quantitative per-sequence evaluation. The scientific significance of the work is that it clarifies when degradation-aware adaptation helps and when it does not. The practical significance is that it provides a reproducible workflow for training and evaluation that can be applied to real-world degraded-image scenarios.

2. RELATED WORK

2.1 DUST3R AS A FOUNDATION FOR 3D RECONSTRUCTION

Recent progress in computer vision has introduced foundation models for geometric tasks. One important example is DUS_t3R [6], which estimates scene geometry from image pairs and supports multi-view 3D reconstruction. Classical pipelines such as COLMAP [5] relies on camera parameters e.g. intrinsic and extrinsic parameters. These are usually tedious and cumbersome to obtain, yet they are mandatory to triangulate corresponding pixels in 3D space, which is the core of all best performing MVS algorithms, whereas DUS_t3R is a radically novel paradigm for Dense and Unconstrained Stereo 3D Reconstruction of arbitrary image collections, i.e. operating without prior information about camera calibration nor viewpoint poses. DUS_t3R provides a deep learning-based formulation that jointly estimates dense correspondences, geometry-related outputs, and relative camera relationships across views. Recent extensions of the DUS_t3R framework further demonstrate the growing importance of foundation models for geometric vision. MAS_t3R [3] extends DUS_t3R with improved matching and geometric consistency for large-scale reconstruction scenarios. MUS_t3R [1] introduces multi-view reasoning strategies that improve robustness and scalability across larger image collections. Fast3R [8] focuses on reducing computational complexity and accelerating inference for practical deployment, while D2US_t3R [2] explores robust dense stereo reconstruction under challenging visual conditions and domain shifts. These developments highlight the rapid evolution of DUS_t3R-based architectures and motivate further research into improving reconstruction robustness under degraded image conditions.

At a high level, DUS_t3R follows an encoder-decoder design. The encoder transforms each image into deep visual tokens and captures both local and global context. A key background for this representation learning is CroCo-style pretraining [7], which improves geometric understanding by learning to recover missing cross-view information from partial observations.

After encoding, DUS_t3R decodes and fuses features from both views to produce geometry-consistent predictions. As shown in Figure 2.2, the decoder uses attention-based blocks for cross-view interaction. In particular,

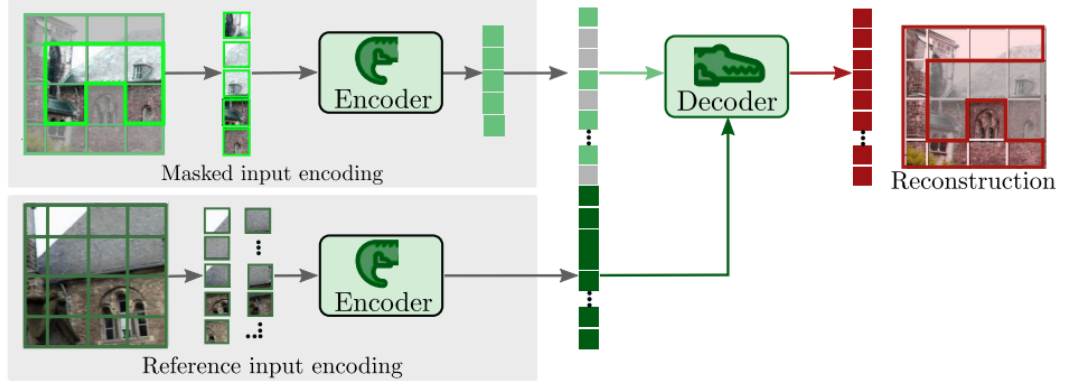


Figure 2.1: Overview of the CroCo architecture (encoder pretraining background).

the CrossBlock alternates self-attention and cross-attention, while the CatBlock concatenates token sets before self-attention. This design helps combine intra-view and inter-view cues effectively.

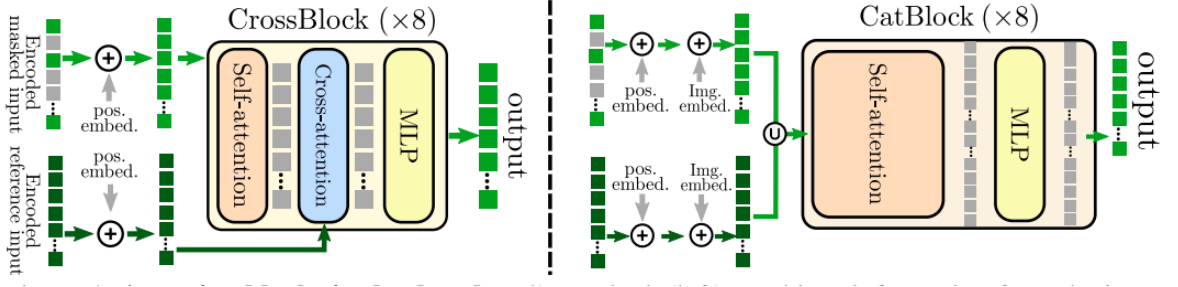


Figure 2.2: Decoder attention design: CrossBlock and CatBlock.

The complete DUST3R pipeline is shown in Figure 2.3. It connects learned visual representations with geometric reasoning for multi-view reconstruction.

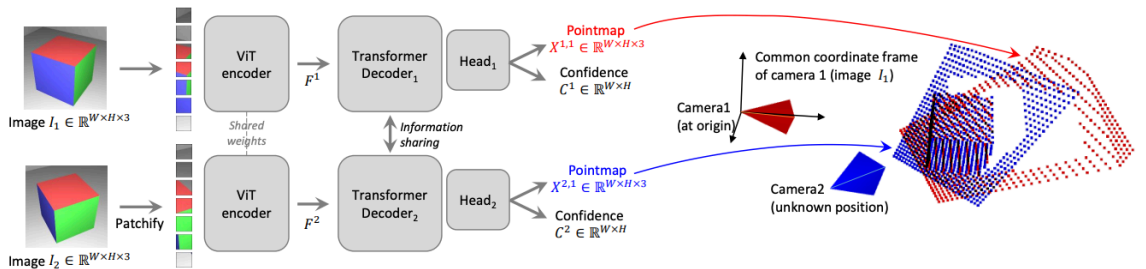


Figure 2.3: Overview of the DUST3R architecture.

Overall, the encoder (with CroCo-related representation learning) and the decoder (with structured attention fusion) form the technical basis of DUST3R. This is the central model family studied in this thesis under Gaussian blur, motion blur, defocus blur.

Evaluation metric. Reconstruction quality is measured with the symmetric Chamfer Distance between predicted and ground-truth point clouds. Let $\mathcal{P} = \{p_i\}_{i=1}^N$ be the predicted point set and $\mathcal{G} = \{g_j\}_{j=1}^M$ be the ground-truth point set. We use:

$$\text{CD}(\mathcal{P}, \mathcal{G}) = \frac{1}{2} \left(\frac{1}{N} \sum_{i=1}^N \min_{g_j \in \mathcal{G}} \|p_i - g_j\|_2^2 + \frac{1}{M} \sum_{j=1}^M \min_{p_i \in \mathcal{P}} \|g_j - p_i\|_2^2 \right). \quad (2.1)$$

Lower values indicate better geometric agreement.

3. METHODOLOGY

3.1 GAUSSIAN BLUR DEGRADATION

Gaussian blur is one of the most common degradations in real capture pipelines. It can be caused by lens limitations focus errors or sensor effects. In mathematical terms, a blurred image can be modeled as a convolution between a clean image and Gaussian kernel:

$$I_{\text{blur}} = I_{\text{clean}} * G_{\sigma},$$

where G_{σ} denotes a Gaussian kernel with standard deviation σ .

For DUS3R-based reconstruction, Gaussian blur is particularly harmful because it weakens local texture and edge information that are required for robust dense correspondence estimation across views. When high-frequency cues are suppressed, matching becomes less stable, which can propagate to less accurate global alignment and noisier point-cloud predictions. Therefore, even when camera viewpoints are sufficient, image blur can reduce the final 3D reconstruction quality.

To mitigate the effect of Gaussian blur, we place a restoration front-end before DUS3R and fine-tune the combined system. We use DeblurDiNAT [4], a recent restoration architecture based on neighborhood attention. In this stage, DUS3R remains frozen, while only DeblurDiNAT parameters are optimized. The architecture of DeblurDiNAT is captured in Figure 3.1.

DeblurDiNAT is selected as the restoration front-end because it is designed not only for strong deblurring accuracy but also for better generalization under challenging and unseen blur patterns. Unlike many deblurring models that mainly optimize distortion metrics (e.g., PSNR/SSIM), DeblurDiNAT emphasizes perceptual quality through its Dilated Neighborhood Attention design, which captures both local and wider blurred structures. Its architecture combines efficient attention, cross-channel interaction, and multi-scale feature fusion (LDFF) in Figure 3.2, making it suitable for recovering details that are important for downstream geometric matching in DUS3R. In addition, DeblurDiNAT is relatively compact compared with heavier restoration backbones, which makes it practical for integration and fine-tuning in our

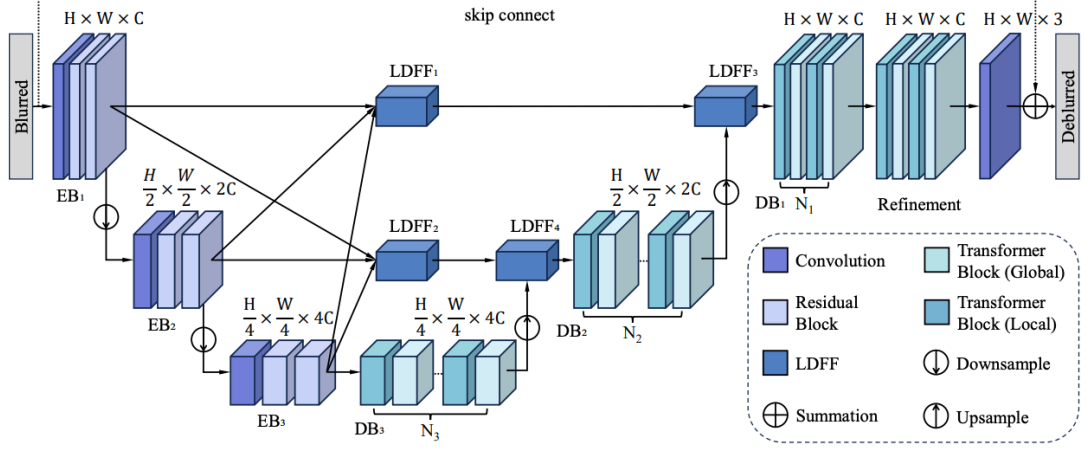


Figure 3.1: Overview of the DeblurDiNAT architecture.

pipeline.

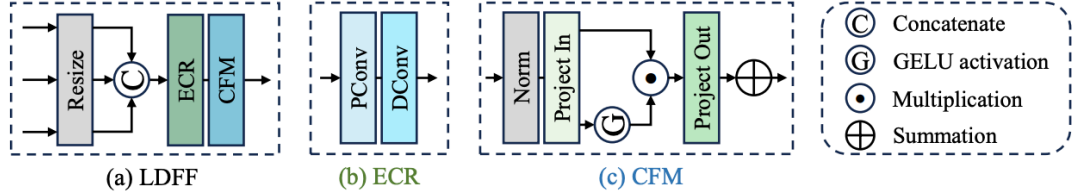


Figure 3.2: Structures of lightweight dual-stage feature fusion (LDFD), composed of efficient channel reduction (ECR) and complementary feature mixer (CFM).

3.1.1 Training objectives

The first joint blur fine-tuning runs were initialized from the DUS3R checkpoint and the DeblurDiNAT checkpoint. The training was performed in `deblurdinat_only` freeze mode, meaning that DUS3R parameters were frozen and only DeblurDiNAT parameters were updated. Two separate joint fine-tuning experiments were conducted: the first used Gaussian blur levels $\sigma \in \{5, 10, 20, 30\}$, while the second used a wider blur range with $\sigma \in \{5, 10, 20, 30, 50\}$.

The first run used a batch size of 1 pair of frames per GPU, 30 epochs, and the AdamW optimizer with a learning rate of 2×10^{-4} and cosine annealing scheduling, weight decay of 0.05, no warmup, a minimum learning rate of $\eta_{\min} = 1 \times 10^{-7}$, and gradient clipping of 1.0. The input resolution was 512×512 , with AMP and gradient checkpointing enabled, and distributed training performed on 4 GPUs.

The second run used the same training configuration, but with 50 epochs and an input resolution of 224×224 . All training experiments were performed

on NVIDIA H100 GPUs.

Notation. In the following, we denote by $X^{n,m}$ the point map X^n from camera n expressed in the coordinate frame of camera m .

Training loss. Our training objective is based on 3D regression with confidence weighting. Let $\bar{X}^{1,1}$ and $\bar{X}^{2,1}$ denote the ground-truth point maps, and let $D_1, D_2 \subseteq \{1, \dots, W\} \times \{1, \dots, H\}$ be the sets of valid pixels. For a valid pixel $i \in D_v$ in view $v \in \{1, 2\}$, the regression loss is:

$$\ell_{\text{regr}}(v, i) = \left\| \frac{1}{z} X_i^{v,1} - \frac{1}{\bar{z}} \bar{X}_i^{v,1} \right\|_2. \quad (3.1)$$

To handle scale ambiguity, predicted and ground-truth point maps are normalized by

$$z = \text{norm}(X^{1,1}, X^{2,1}), \quad \bar{z} = \text{norm}(\bar{X}^{1,1}, \bar{X}^{2,1}),$$

where

$$\text{norm}(X^1, X^2) = \frac{1}{|D_1| + |D_2|} \sum_{v \in \{1,2\}} \sum_{i \in D_v} \|X_i^v\|_2. \quad (3.2)$$

Since some pixels are harder or ill-defined (e.g., sky or translucent regions), the model predicts confidence scores $C_i^{v,1}$ and uses the confidence-aware objective:

$$\mathcal{L}_{\text{conf}} = \sum_{v \in \{1,2\}} \sum_{i \in D_v} (C_i^{v,1} \ell_{\text{regr}}(v, i) - \alpha \log C_i^{v,1}), \quad (3.3)$$

where α is a hyper-parameter controlling the regularization term.

3.1.2 Data Description

For Gaussian-blur fine-tuning, we used the processed 10-category CO3D subset together with its synthetically degraded counterpart. Training was performed jointly over multiple blur levels in a single run, with Gaussian kernel standard deviations $\sigma \in \{5, 10, 20, 30\}$, by concatenating samples from all selected degradation levels.

The category set follows the 10-category CO3D configuration used in this work: apple, banana, baseballbat, baseballglove, bicycle, bowl, broccoli, cake, car, and carrot. A total of 8 sequences were selected for each category.

Data partitioning in this experiment was performed using a random sample-level split. Specifically, all available training samples were first

pooled and then randomly shuffled using a fixed seed, after which the indices were divided into training, validation, and test subsets according to predefined ratios of 80%/10%/10%. Consequently, samples from the same sequence may appear in different subsets. This design was chosen to enable efficient optimization during restoration front-end fine-tuning.

3.1.3 Results

Test categories and protocol. Evaluation was performed on six CO3D categories: bottle, cup, donut, teddybear, couch, and toytrain. For each category, one fixed sequence was used for testing in this Gaussian-blur benchmark (i.e., 6 sequences in total). Chamfer Distance was first computed per sequence, then averaged across the six test sequences. Therefore, all values reported in Tables ?? and ?? are **average Chamfer Distance** values (lower is better). The results demonstrate that the pretrained DeblurDiNAT front-end

Table 3.1: Average Chamfer Distance comparison for Gaussian blur levels $\sigma = 5, 10, 20, 30$ (lower is better). It is used DUS3R-ViTLarge-BaseDecoder-224-linear.pth

Methods	$\sigma = 5$	$\sigma = 10$	$\sigma = 20$	$\sigma = 30$
DUS3R-Pretrain	1.4611	1.6993	2.2381	NaN
DeblurDiNAT-Pretrain + DUS3R	1.4293	1.6778	1.9284	2.0382
DeblurDiNAT-Finetuned + DUS3R	1.7516	1.6508	1.6454	1.6519

Table 3.2: Average Chamfer Distance comparison for Gaussian blur levels $\sigma = 5, 20, 30, 50$ (lower is better). It is used DUS3R-ViTLarge-BaseDecoder-224-linear.pth

Methods	$\sigma = 5$	$\sigma = 20$	$\sigma = 30$	$\sigma = 50$
DUS3R-Pretrain	1.4607	2.2348	NaN	1.9091
DeblurDiNAT-Pretrain + DUS3R	1.4292	1.9270	1.9057	2.0372
DeblurDiNAT-Finetuned + DUS3R	2.8661	1.8080	1.7367	1.7731

already improves reconstruction robustness compared to the original DUS3R baseline under strong Gaussian blur. However, the jointly fine-tuned DeblurDiNAT + DUS3R model achieves the best performance across all evaluated blur levels. The improvement becomes more significant as the blur intensity increases, indicating that joint fine-tuning helps the reconstruction pipeline adapt more effectively to severe image degradation conditions.

3.2 MOTION AND DEFOCUS BLUR DEGRADATION

Real captures often violate the sharp-image assumption through motion blur (integration along the camera or scene motion) and defocus blur (out-of-focus regions behaving like convolution with a disk-shaped point spread). To improve downstream 3D reconstruction without modifying DUS3R in this stage, we train a lightweight image restoration front-end using knowledge distillation from strong, frozen Restormer [9] models specialized for motion and defocus.

To obtain high-quality restoration targets without training a large model from scratch, we adopt Restormer as a frozen teacher. Restormer is a Transformer-based general restoration architecture with strong published results on motion deblurring and defocus deblurring; official checkpoints are available for both tasks. We use two Restormer instances with the same architecture but different weights: one checkpoint trained for motion blur, one for single-image defocus deblurring. Given a corrupted frame, the appropriate branch produces the teacher output. Restormer is chosen because it is strong and well documented, matches our two corruption types, and allows a fixed, reproducible teacher without collecting new datasets.

Because running Restormer before DUS3R at full resolution is heavy, we train a small student restoration network to mimic the teacher outputs under the same synthetic corruptions (knowledge distillation). DUS3R is not trained in this stage; the student is trained so that, in later evaluation, restored images can be fed to the unchanged DUS3R model.

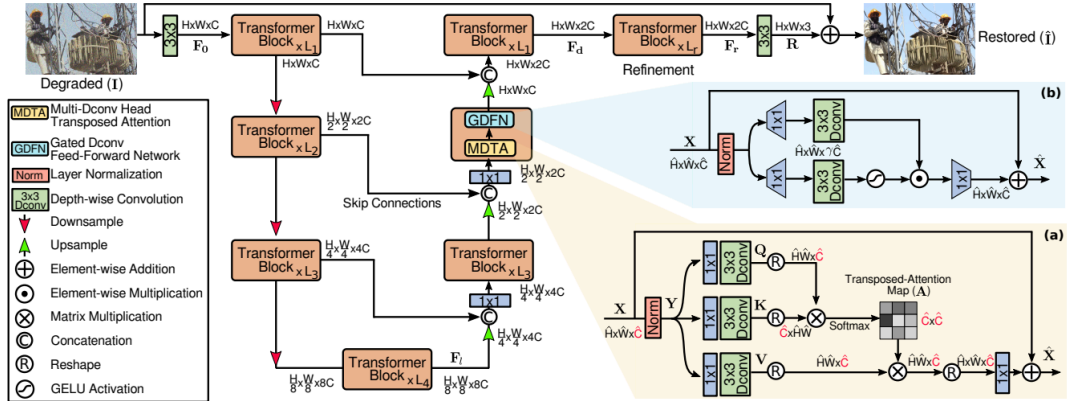


Figure 3.3: Architecture of Restormer for high-resolution image restoration. Our

Restormer consists of multiscale hierarchical design incorporating efficient Transformer blocks. The core modules of Transformer block are: (a) multi-Dconv head transposed attention (MDTA) that performs (spatially enriched) query-key feature interaction across channels rather the spatial dimension, and (b) Gated-Dconv feed-forward network (GDFN) that performs controlled feature transformation, i.e., to allow useful information to propagate further.

3.2.1 Training objectives

Distillation is supervised in image space against the teacher. For each corrupted input x , the teacher T (motion or defocus branch selected by the corruption type) produces a target $y = T(x)$. The student S_θ predicts $\hat{y} = S_\theta(x)$. Training minimizes a masked combination of ℓ_1 and SSIM-based terms so that variable-size batches remain valid where padding is used:

$$L_{\text{total}} = L_1(\hat{y}, y) + 0.2 \cdot (1 - \text{SSIM}(\hat{y}, y)),$$

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

where μ_x and μ_y denote the local mean intensities of images x and y , σ_x^2 and σ_y^2 are the corresponding local variances, and σ_{xy} is the local covariance between the two images. The constants C_1 and C_2 are small positive values introduced to stabilize the division in regions with low intensity variance.

With L_1 defined as the mean absolute error averaged over valid pixels (and channels) inside a binary mask, and SSIM computed using a standard Gaussian window with the same masking. The teacher is used under no gradient ($\nabla T \equiv 0$); only θ is updated.

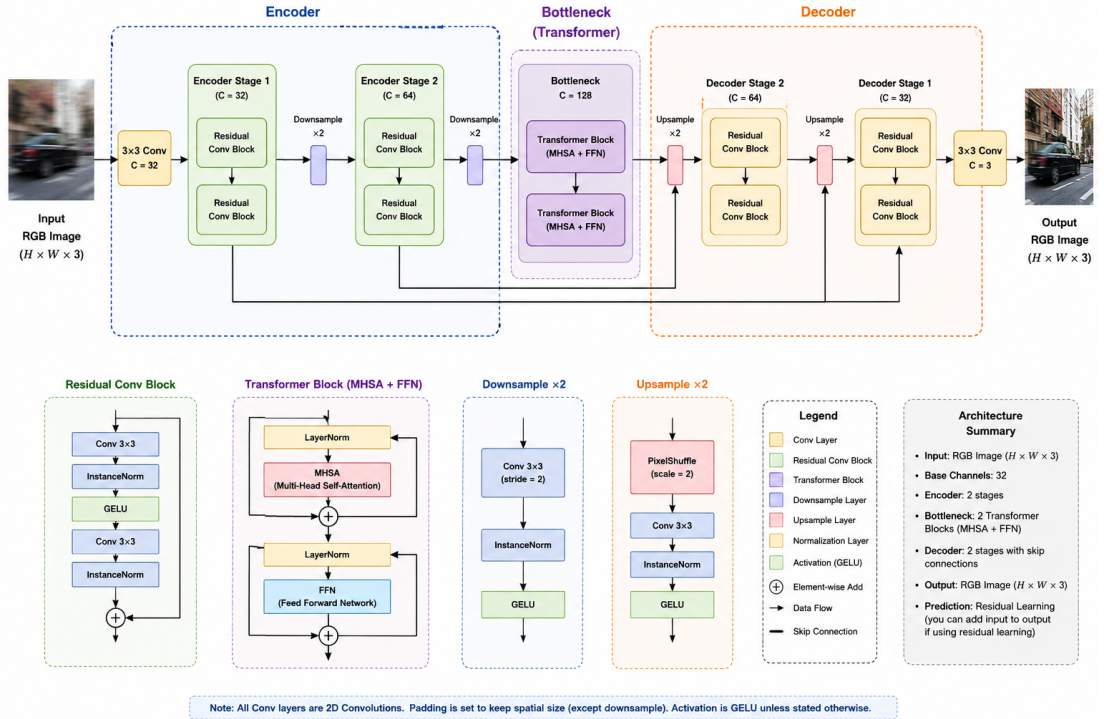


Figure 3.4: Overview of the Student architecture

We performed two KD Restormer training runs. The first run used the AdamW optimizer with weight decay 10^{-4} and default momentum parameters (0.9, 0.999). Training was conducted for 20 epochs, with the learning rate initialized at 3×10^{-4} and cosine annealed down to 10^{-6} . Motion and defocus degradations were applied using `motion_kernel_size` = 25 and `defocus_radius` = 7, respectively. A batch size of 4 samples per GPU was used; since training was performed on 2 GPUs, the effective total batch size was 8.

The second training run used more severe motion and defocus blur settings, with `motion_kernel_size` = 35 and `defocus_radius` = 9, respectively. This run was performed on a single GPU with an effective batch size of 4, 50 epochs. Other hyperparameters are the same.

Both the first and the second training has an implementation such that each epoch consisted of a motion training phase over all motion-corrupted training frames, followed by a defocus training phase over all defocus-corrupted training frames, ensuring that both degradations contributed during every epoch. Validation reported motion and defocus losses separately, and the best student checkpoint was selected based on the mean of the two validation losses.

We also fine-tune a DeblurDiNAT restoration network placed in front of a pretrained DUS3R model. By default, only the DeblurDiNAT parameters are trainable, while DUS3R remains frozen, as in the Gaussian blur setting. Blur degradations are generated dynamically rather than loaded from disk. Motion blur is implemented using a normalized 25×25 horizontal line kernel with ones along the center row, while defocus blur uses a normalized circular disk kernel of radius 7 defined on a 15×15 support. For each sample in a batch, motion blur is selected with probability `motion_prob` (default value 0.5), while defocus blur is applied otherwise. Regarding the train/validation dataset split, a random sample-level split of image pairs is performed, as described in Section 3.1. Optimization is performed using AdamW on the trainable parameters, with default settings including a learning rate of 10^{-4} , weight decay of 0.05, and gradient clipping of 1.0. Mixed-precision training and optional gradient checkpointing are enabled for both DUS3R and DeblurDiNAT. The learning-rate schedule follows cosine decay down to 10^{-7} . All training experiments were performed on NVIDIA H100 GPUs.

3.2.2 Data description

The experiment are restricted to the same ten categories. The train, validation split is sequence-level all frames belonging to sequences assigned to training are used for optimization, and all frames belonging to sequences assigned to validation are used for monitoring and checkpoint selection, without mixing the two sets. At sample time, a frame is read in BGR layout, synthetically blurred, and converted to a floating-point RGB tensor in $[0, 1]$. Two mutually exclusive training regimes are instantiated as separate datasets: one dataset applies only *motion* blur, the other only *defocus* blur. Motion blur uses a normalized horizontal line kernel on a 25×25 grid (middle row equal to one, all other entries zero, then divided by the sum of entries); defocus blur uses a normalized binary disk of radius seven pixels on a 15×15 support. Both filters are applied with OpenCV `filter2D`. During the first training, each time a frame is drawn for an update, the corruption type is selected independently with equal probability: with probability one-half the frame is taken from the motion-blurring pipeline, and with probability one-half from the defocus-blurring pipeline, so motion and defocus degradations contribute symmetrically in expectation over the course of optimization. In contrast, during the second training run, we constructed two separate datasets: one containing only motion-blurred samples and another containing only defocus-blurred samples. Each epoch consisted of first iterating through the entire motion-blur training loader, followed by the entire defocus-blur training loader. The same protocol was used for validation.

3.2.3 Results

Evaluation was performed on six CO3D categories: bottle, cup, donut, teddybear, couch, toytrain. In contrast, DeblurDiNAT (finetuned) + DUST3R is evaluated on bottle, couch, cup, donut, hydrant, teddybear, toybus, toytrain. For every such sequence, we subsampled 20 RGB frames, applied the protocol in Section 3.2 (clean, synthetic motion blur, or synthetic defocus blur), ran DUST3R with global alignment, and compared the fused point cloud to the ground-truth pointcloud using Chamfer distance. Chamfer distance was computed per sequence, then averaged across all evaluated sequences to obtain the scalar reported for each scenario. Therefore, the entries in Table 3.4 are mean Chamfer distances (lower is better).

Table 3.3: Average Chamfer distance on the CO3D test split for clean, motion-blurred, and defocus-blurred inputs (lower is better). Motion blur: normalized horizontal line kernel on a 25×25 support; defocus blur: normalized disk of radius 7 on a 15×15 support. It is used DUST3R-ViTLarge-BaseDecoder-512-dpt.pth

Methods	Clean	Motion blur	Defocus blur
DUST3R	1.5662	1.6006	1.5917
KD student + DUST3R	1.5689	1.5854	1.5766

By contrast, under synthetic motion blur and synthetic defocus blur, the student-augmented pipeline achieves lower average Chamfer than the vanilla DUST3R baseline, indicating that Restormer-guided distillation improves robustness of the downstream reconstruction stage to these degradations. The gains are modest in absolute terms but consistent across the two blur families, suggesting that the student learns restoration cues that better align multi-view geometry estimation with DUST3R’s pretrained stereo prior when blur is present.

Table 3.4: Average Chamfer distance on the CO3D test split for clean, motion-blurred, and defocus-blurred inputs (lower is better). Motion blur: normalized horizontal line kernel on a 35×35 support; defocus blur: normalized disk of radius 9 on a 19×19 support. DUST3R is initialized from DUST3R_ViTLarge_BaseDecoder_512_dpt.pth.

Methods	Clean	Motion blur	Defocus blur
DUST3R	1.5662	1.6006	1.5917
KD student + DUST3R	1.5654	1.5944	1.5805

Table 3.4 shows that the second training of the Restormer-KD student slightly improves mean Chamfer on clean inputs and reduces it on both synthetic motion and defocus blur relative to DUST3R alone, using the same ViT-Large 512 DUST3R checkpoint in all rows.

Table 3.5: Average Chamfer distance on the CO3D test split for clean, motion-blurred, and defocus-blurred inputs (lower is better). Motion blur: normalized horizontal line kernel on a 25×25 support; defocus blur: normalized disk of radius 7 on a 15×15 support. It is used DUST3R-ViTLarge-BaseDecoder-512-dpt.pth

Methods	Clean	Motion blur	Defocus blur
DUST3R	1.5949	1.6312	1.6192
DeblurDiNAT (finetuned) + DUST3R	1.6113	1.6014	1.6146

Table 3.5 compares vanilla DUST3R to the pipeline with a fine-tuned DeblurDiNAT front-end under clean, synthetic motion-blurred, and synthetic defocus-blurred inputs on the CO3D test split (eight categories; mean Chamfer over all evaluated sequences). The finetuned deblurring module slightly increases Chamfer on clean frames, but reduces it on both blur types, with

the largest gain under motion blur. All runs use the same pretrained DUS_t3R backbone and alignment budget stated in the caption.

CONCLUSIONS AND RECOMMENDATIONS

In this thesis, we investigated the robustness of DUS_t3R-based 3D reconstruction under different image degradations, with a particular focus on Gaussian blur, motion blur, and defocus blur. Since dense geometric matching strongly depends on high-frequency visual details, image degradation can significantly reduce reconstruction accuracy and stability. To address this limitation, we explored restoration front-end strategies that improve image quality before reconstruction while preserving the original DUS_t3R architecture.

For Gaussian blur, we integrated DeblurDiNAT as a restoration module placed before a frozen DUS_t3R backbone and performed joint fine-tuning under multiple synthetic blur levels. Experimental results on CO3D demonstrated that the pretrained DeblurDiNAT already improves reconstruction robustness compared to vanilla DUS_t3R, while joint fine-tuning further reduces Chamfer Distance across all evaluated blur intensities. The improvement became more pronounced for stronger blur levels, showing that the restoration front-end successfully adapts the degraded inputs to the geometric priors learned by DUS_t3R.

For motion and defocus blur, we adopted a knowledge-distillation framework using strong pretrained Restormer models as frozen teachers. A lightweight student restoration network was trained to mimic the teacher outputs under synthetic degradations. The distilled student, when combined with DUS_t3R, consistently improved reconstruction quality under both motion and defocus blur while maintaining nearly identical performance on clean images. We additionally fine-tuned DeblurDiNAT under dynamically generated motion and defocus degradations, which further improved robustness, especially under stronger motion blur conditions.

Overall, the experiments demonstrate that restoration-aware preprocessing substantially improves the robustness of DUS_t3R-based multi-view 3D reconstruction under realistic image degradations. The results indicate that even lightweight restoration modules can effectively recover image structures that are important for reliable dense correspondence estimation and global alignment.

Several directions remain for future work. One possibility is to jointly optimize both the restoration network and DUS_t3R end-to-end instead of freez-

ing the reconstruction backbone. Another direction is to investigate additional real-world degradations such as image noise, low-light conditions, compression artifacts, and combined degradations. Finally, extending the framework toward real captured datasets and video-based reconstruction scenarios could further improve practical robustness in unconstrained environments.

REFERENCES

- [1] Yohann Cabon et al. “MUST3R: Multi-view Network for Stereo 3D Reconstruction”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2025.
- [2] Jisang Han et al. “D2USt3R: Robust Dense and Unconstrained Stereo 3D Reconstruction”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2025.
- [3] Vincent Leroy et al. “MASt3R: Matching and Stereo 3D Reconstruction”. In: *Proceedings of the European Conference on Computer Vision (ECCV)*. 2024.
- [4] Hanzhou Liu et al. “DeblurDiNAT: A Compact Model with Exceptional Generalization and Visual Fidelity on Unseen Domains”. In: *arXiv preprint arXiv:2403.13163* (2025).
- [5] Johannes L. Schönberger and Jan-Michael Frahm. “Structure-from-Motion Revisited”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016.
- [6] Shuzhe Wang et al. “DUSt3R: Geometric 3D Vision Made Easy”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2024.
- [7] Philippe Weinzaepfel et al. “CroCo: Self-Supervised Pre-training for 3D Vision Tasks by Cross-View Completion”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 2023.
- [8] Jianing Yang et al. “Fast3R: Towards Fast and Efficient 3D Reconstruction”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2025.
- [9] Syed Waqas Zamir et al. “Restormer: Efficient Transformer for High-Resolution Image Restoration”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2021.